

Hospital Triage AI Multiagente

Programa web funcional de triage inteligente con agentes visuales, orquestación multiagente, RAG y MCP local.

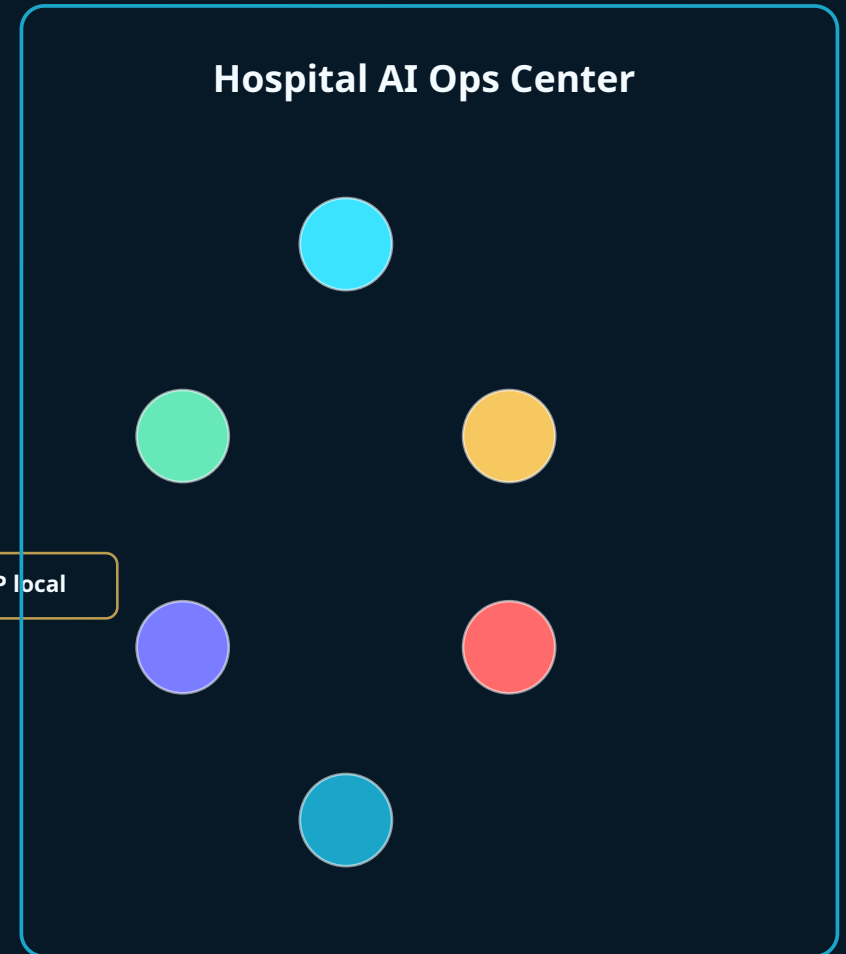
Next.js · React · TypeScript

FastAPI · LangGraph

MongoDB · Elasticsearch

MCP local

Objetivo: construir una aplicación ejecutable, no una maqueta estática.



Resumen ejecutivo

Qué se quiere construir y por qué tiene valor

Producto

Aplicación web funcional para apoyar triage en urgencias: entrada de paciente, agentes visuales, trazabilidad y resultado final.

Diferencial

No solo responde un modelo: se ve cómo un orquestador coordina subagentes especializados en tiempo real.

Límite clínico

No diagnostica. Entrega prioridad sugerida, factores de riesgo y próximos pasos para revisión humana.

Idea fuerza

La inteligencia del sistema no está solo en el modelo, sino en cómo se organiza el trabajo: orquestación, especialización, trazabilidad y validación humana.

Alcance del MVP

Lo que entra, lo que queda fuera y por qué

Incluido

- Frontend Next.js + React + TypeScript
- Escenario hospitalario pixelado con agentes visibles
- Backend FastAPI + LangGraph + LangChain
- MongoDB para casos, trazas, estados e informes
- Elasticsearch para protocolos y RAG
- MCP local de alcance controlado para herramientas hospitalarias simuladas
- Docker Compose para ejecución local

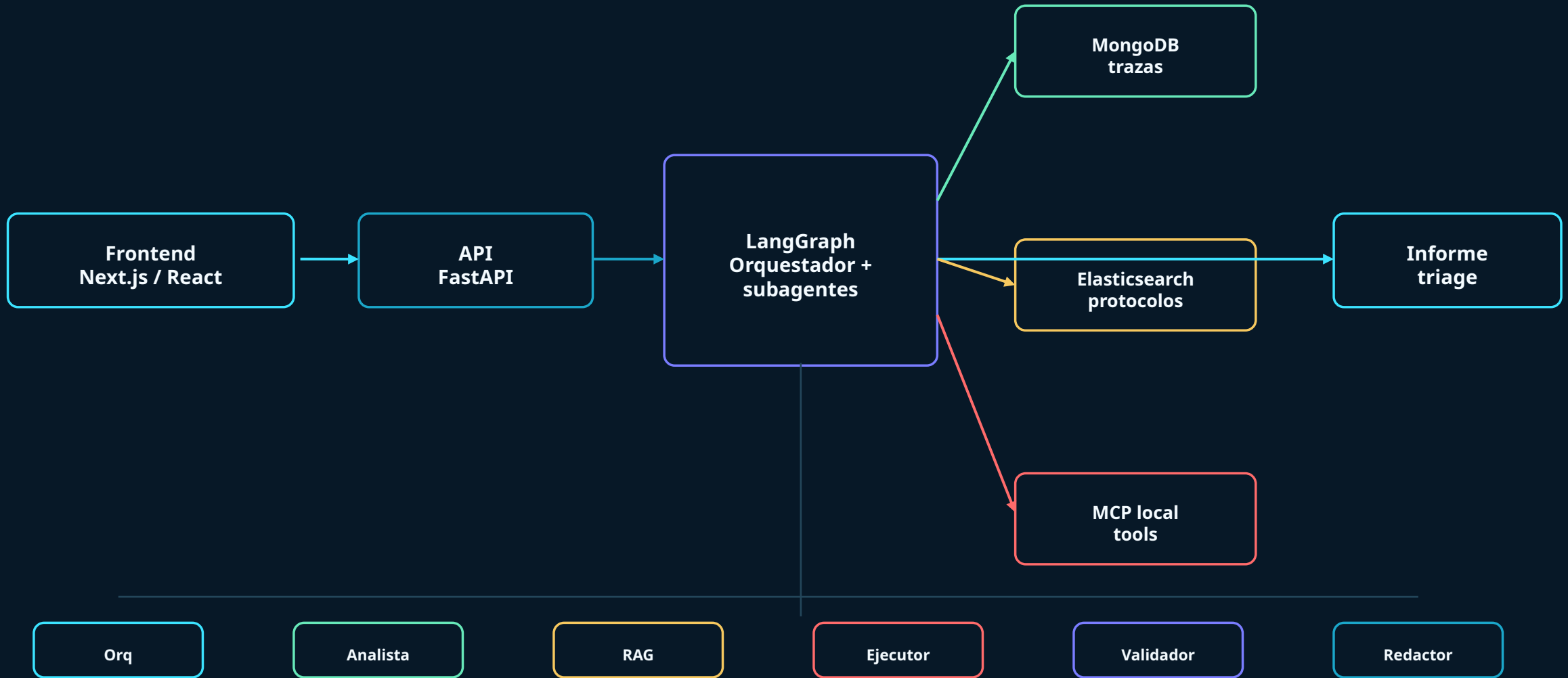
Fuera o limitado

- No diagnóstico automático
- No Streamlit como frontend principal
- No SQL/PostgreSQL salvo petición posterior
- No Qdrant, Chroma, Supabase ni cloud no solicitado
- No convertirlo en proyecto OpenClaw
- No arquitectura sobredimensionada antes de validar el flujo

OpenClaw Studio se usa solo como referencia visual/cantera de pixel_agent. El proyecto final es standalone: hospital-triage-ai.

Arquitectura funcional y técnica

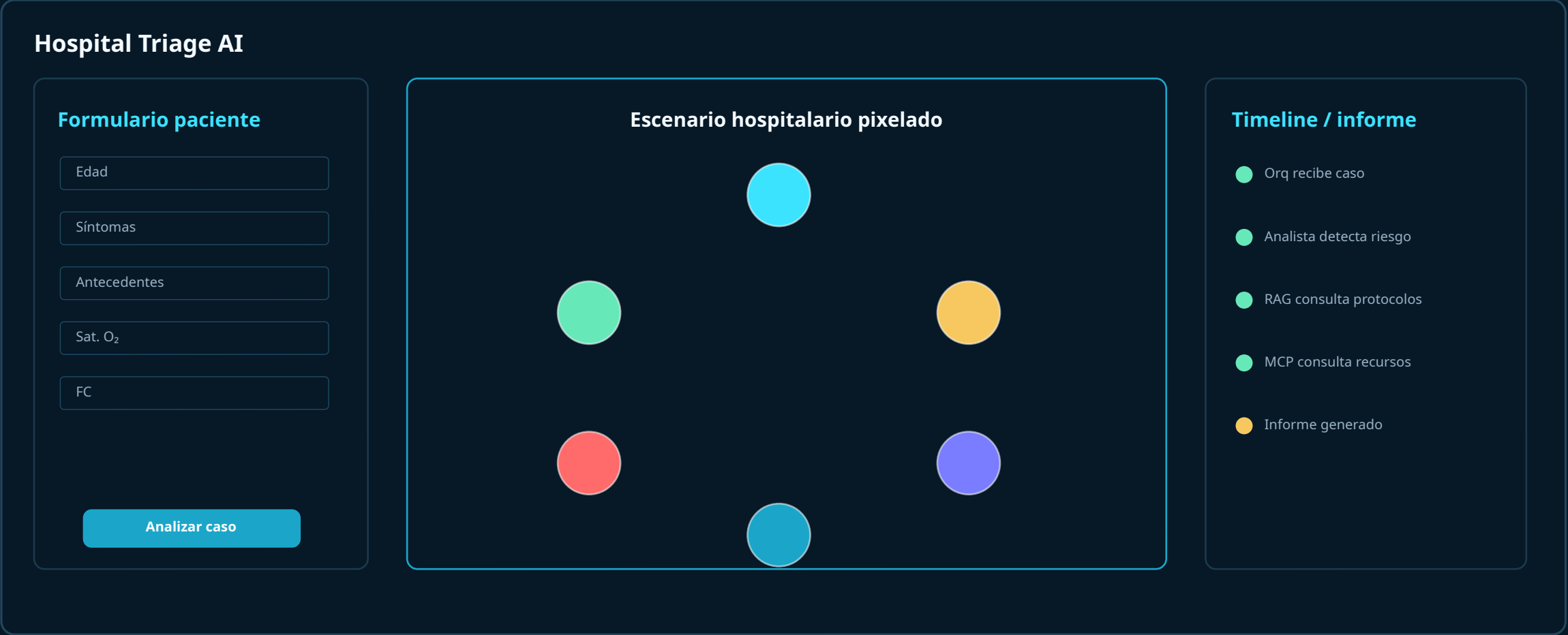
Separación clara entre UI, API, agentes, datos y herramientas



Los eventos del grafo alimentan la visualización de estados de los agentes.

Experiencia de usuario

El escenario hospitalario es el protagonista



Una única pantalla: formulario compacto + escena central viva + trazabilidad.

Agentes y responsabilidades

Cada agente tiene una tarea clara y visible



Coordinador

Decide el flujo y delega tareas

receiving_case / thinking



Analista clínico

Detecta riesgo en síntomas y constantes

analyzing



Investigador RAG

Recupera protocolos con Elasticsearch

searching



Ejecutor MCP

Consulta herramientas hospitalarias simuladas

executing



Validador

Revisa seguridad y límites clínicos

validating



Redactor

Genera informe preliminar final

writing / completed

Estados mínimos: idle · receiving_case · thinking · walking · analyzing · searching · executing · validating · writing · completed · blocked · error

Observabilidad visual de IA

No es estética de videojuego: es supervisión operativa

Cada agente visual representa el estado operativo de una entidad autónoma del sistema. La representación visual permite supervisión humana en tiempo real, trazabilidad de tareas y detección temprana de fallos o consumo ineficiente de recursos.



Quieto

Bloqueo



Dando vueltas

Loop



Muchos en zona

Cuello de botella



Rojo

Error / token limit



Amarillo

Esperando respuesta



Verde

Tarea completada

Valor enterprise: detectar loops, esperas, bloqueos, errores y sobreuso de LLM antes de que el operador tenga que leer logs eternos.

MCP en el MVP

Servidor local de alcance controlado, útil y defendible

Qué será

MCP Server local para exponer herramientas hospitalarias simuladas al flujo multiagente.

Quién lo usa

Principalmente hospital_systems_executor para consultar contexto operativo.

Qué evita

No reemplaza FastAPI, LangGraph, MongoDB ni Elasticsearch. Es una integración acotada.

`get_available_tests`

Disponibilidad de pruebas

`get_resource_status`

Boxes, oxígeno y recursos

`get_wait_time`

Tiempo estimado de espera

`get_operational_context`

Contexto operativo simulado

Debe tener fallback: si MCP falla, la demo no se rompe; se informa en la traza y continúa con datos simulados.

Datos, RAG y persistencia

MongoDB y Elasticsearch juntos, pero no mezclados



Regla: MongoDB no es motor documental principal; Elasticsearch no es base principal de casos.

Plan de implementación por fases

Trabajo incremental para reducir riesgos



Estrategia: construir, validar y probar cada fase antes de avanzar. Nada de "todo a la vez" con fe y cafeína.

Pruebas y definición de terminado

Lo mínimo para considerar el MVP listo



Backend

/triage, validación de entrada y disclaimer



LangGraph

ejecución de nodos y agent_trace



MongoDB

guardar y recuperar casos e informes



Elasticsearch/RAG

indexar protocolos y recuperar fragmentos



MCP

tools locales y fallback controlado



Frontend

formulario, agentes, estados y timeline



Integración

frontend llama backend y muestra progreso



Docker

servicios levantan localmente

Terminado = aplicación local ejecutable + flujo multiagente visible + informe seguro + datos/trazas persistidos + pruebas pasadas.

Contexto operativo y siguiente paso

Rutas de referencia y arranque controlado

Ruta del proyecto

C:\Users\0021260\OneDrive - ViewNext\Escritorio\VisualStudio\viewnext-ai-course\hospital-triage-ai

Referencia visual

C:\Users\0021260\OneDrive - ViewNext\Escritorio\VisualStudio\viewnext-ai-course\openclaw-studio

Referencia LangGraph agentes

...\LangGraf\agentes_reskilling\tarea_caso_de_uso

Referencias curso

FastAPI · LangChain · LangGraf · NoSQL · RAG · Bases vectoriales

Siguiente paso propuesto: aprobar alcance, crear repositorio privado, ejecutar fase 0 de análisis y validar el plan técnico antes de tocar código.

Hospital Triage AI Multiagente · MVP seguro, visual y funcional para mostrar en directo

12. Actualización: informe mediante Google Drive o Gmail

Recomendación del tutor integrada en el alcance del MVP

El MVP incorporará una salida externa para el informe preliminar generado por el flujo multiagente. El resultado no quedará únicamente en pantalla: podrá guardarse en Google Drive o enviarse por Gmail, según el alcance técnico que se active durante la implementación.

La opción recomendada es priorizar Google Drive como destino principal, porque permite demostrar generación documental, trazabilidad del resultado y consulta posterior del informe. Gmail puede quedar como alternativa para enviar el resumen o el documento a un destinatario simulado.

Qué aporta

- Informe externo y revisable
- Trazabilidad documental
- Mejor valor práctico del programa
- Resultado útil más allá de la UI

Implementación controlada

- `save_triage_report_to_drive`
- `send_triage_report_email` opcional
- Fallback/mock si no hay credenciales
- Estado visible en timeline e informe

Integración con MCP: esta capacidad puede exponerse mediante el MCP local de alcance controlado o una capa de servicio desacoplada. No sustituye a FastAPI, LangGraph, MongoDB ni Elasticsearch. El informe debe generarse desde `report_writer`, registrar la trazabilidad en MongoDB y mantener el aviso de seguridad clínica.